Task 1: 계좌 통합 관리

연구실의 각 프로젝트 계좌를 합치거나 예산을 지출하는 시스템을 설계해 봅시다.

Mission: Account 클래스에 산술 연산 기능 추가하기

1. **초기화:** `name`(계좌명)과 `balance`(잔액)를 속성으로 가집니다.
2. **더하기 오버로딩 (__add__):** 두 계좌를 더하면 잔액은 합산되고, 이름은 "A & B" 형태로 결합된 **새로운 Account 객체**를 반환하세요.
3. **빼기 오버로딩 (__sub__):** 계좌에서 숫자(정수)를 빼면 잔액이 그만큼 차감되도록 구현하세요. (예: `acct - 5000`)
4. **출력 설정 (__str__):** `print()` 호출 시 "계좌[이름]: 잔액원" 형식으로 출력되게 하세요.

실행 결과 예시

```
a1 = Account("민석 과제", 100000)
a2 = Account("용준 과제", 200000)
a3 = a1 + a2        # 계좌[민석 과제 & 용준 과제]: 300000원
a3 = a3 - 50000    # 계좌[민석 과제 & 용준 과제]: 250000원
```

Task 2: 나만의 게임 캐릭터 불러오기

여러 클래스를 별도의 파일(모듈)로 나누어 관리하고, 게임 서버에서 이를 불러오는 연습을 해봅시다.

Mission: 캐릭터 모듈 생성 및 임포트

1. **파일 1 (game_chars.py):** `Warrior` 클래스와 `Mage` 클래스를 정의하세요.
2. **파일 2 (main.py):** `from game_chars import Warrior`를 사용하여 전사만 가져와 인스턴스를 만드세요.
3. **별칭 연습:** `import game_chars as gc`를 사용하여 `gc.Mage()`로 마법사 인스턴스를 만드세요.
4. **검증:** 각 캐릭터의 공격(`attack`) 메서드를 호출하여 정상적으로 출력되는지 확인합니다.

팁

리그 오브 레전드 같은 큰 게임도 수백 명의 챔피언 정보를 이런 식으로 모듈화해서 관리합니다. 코드가 깔끔해지는 마법입니다.

Task 3: RPG 캐릭터 전직 시스템 설계

캐릭터가 전직할 때마다 이전 단계의 기술을 그대로 간직하며 새로운 기술을 배우는 '상속의 흐름'을 구현해 봅시다.

Mission: 3단계 전직 구조 만들기

1. **GameCharacter (초보자):** `move()` 메서드 ("기본 이동을 합니다")
2. **Warrior (전사):** `GameCharacter`를 상속받고, `slash()` 메서드 ("칼을 휘두릅니다")
3. **Berserker (광전사):** `Warrior`를 상속받고, `rage()` 메서드 ("폭주 상태가 됩니다!")
4. **실험:** `Berserker` 인스턴스를 하나 만들고, 초보자 기술(`move`), 전사 기술(`slash`), 광전사 기술(`rage`)을 모두 호출해 보세요.